Integrated MSc(IT)

UNIT – 2

Artificial Intelligence: An Introduction

221601601 ARTIFICIAL INTELLIGENCE

Defining Problem as State Space Search

- In solving problems, each possible ways of solving problem must be considered.
- Also, each possible combinations of factors influencing problem solving should be considered.
- Many problems can be formalised in a general way as search problems.

Analyzing Problem

- AI Problem can be is classified as:
  - Mundane/ General Tasks
  - Formal Tasks
  - Expert Tasks

Analyzing Problem

- **Mundane Tasks:**
  - These are the ones that the humans do on regular basis without any special training such as computer vision, speech recognition, Natural language processing, generation and translation etc.
  - Common sense, reasoning and planning are the common characteristics of these tasks.

- **Formal Tasks:**
  - These are the ones where there is an application of formal logic, some learning etc.
  - Verifications, Theorom proving etc are the common characteristics.
  - Games such as Chess Checkers, Go etc are classified in these task.

- **Expert Tasks:**
  - These are which comes under functional expert domain such as engineering, fault finding, manufacturing planning, medical diagnosis etc.
  - These Tasks which requires high analytical and thinking skills, a job only a professionals can do.

Isolating Task Oriented Knowledge

- Solving the problem, knowledge related to the field is required.
- As per the nature of AI applications, knowledge and there representation varies and so does the representation techniques.
- Various knowledge representation (KR) techniques are designed in this phase.

Finding the Solution

- After representation of problem and knowledge in suitable format, the appropriate methodology is chosen which uses knowledge and transforms the start goal to end goal.
- The techniques of finding the solution are called search techniques.

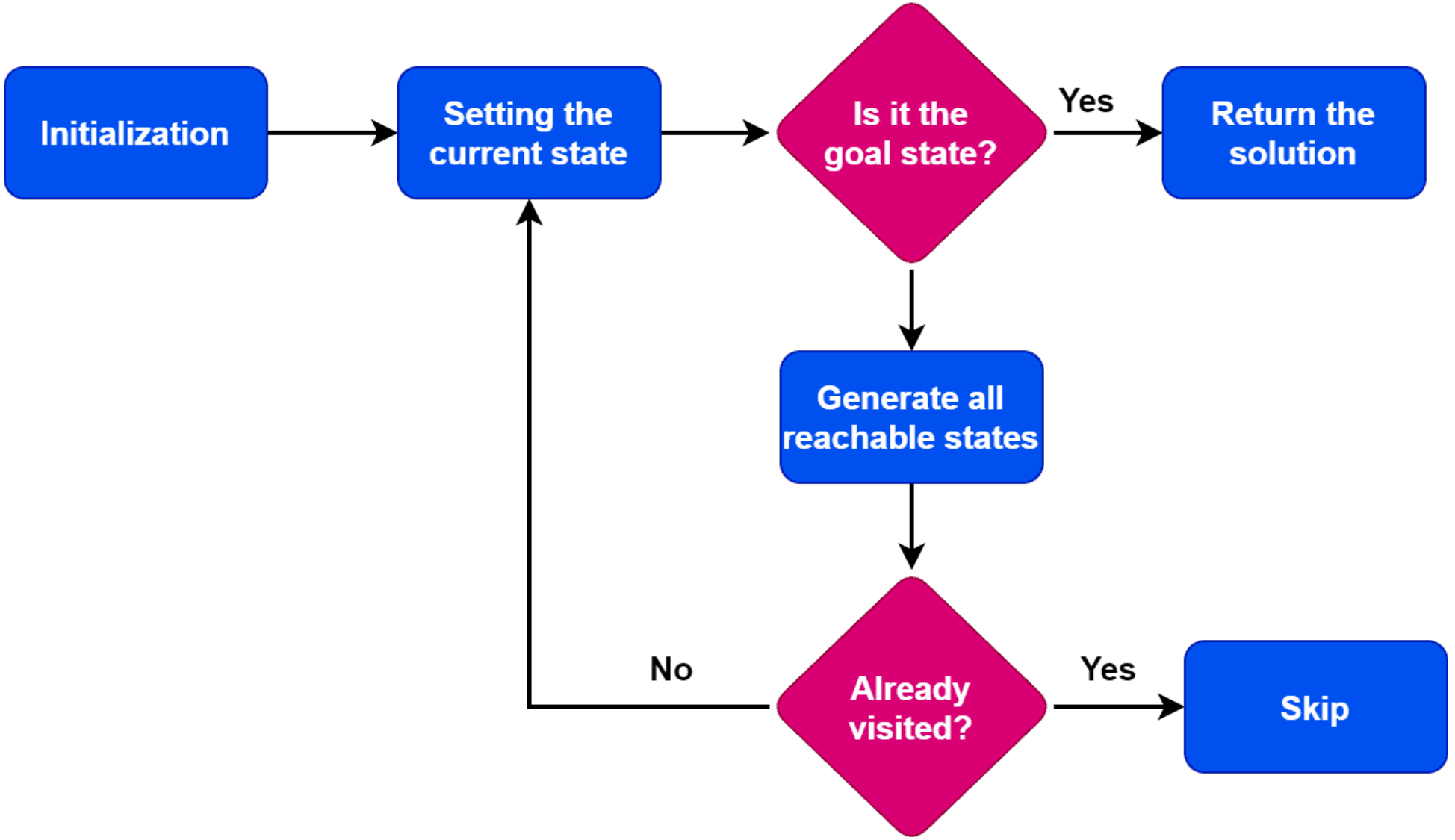
Representation of AI problem as State Space Search

- **State Space Search** is a problem-solving technique used in Artificial Intelligence (AI) to find the solution path from the initial state to the goal state by exploring the various states.
- The state space search approach searches through all possible states of a problem to find a solution.
- It is an essential part of Artificial Intelligence and is used in various applications, from game-playing algorithms to natural language processing.
- By modeling problems as states and transitions, AI systems can solve complex tasks like puzzle-solving, robotics, and planning through systematic search strategies that efficiently navigate toward the goal state.

Representation of AI problem as State Space Search

- A state space is a way to **mathematically represent** a problem by defining all the possible states in which the problem can be.
- This is used in search algorithms to represent the
  - **Initial State**
  - **Goal State**
  - **Current State** of the problem.
- Each state in the state space is represented using a set of variables.
- The efficiency of the search algorithm greatly depends on the size of the state space, and it is important to choose an appropriate representation and search strategy to search the state space efficiently.

State Space Search Algorithm



State Space Search Algorithm

- **Step 1: Define the State Space**
  - The first step in state space search is to define the state space clearly.
  - This involves identifying all possible states the system can be in and the transitions between these states.
  - For example, in the 8-puzzle problem, each configuration of the puzzle represents a state, and moving a tile constitutes a transition.
  - Defining the state space accurately is critical for setting up the search process.

State Space Search Algorithm

- **Step 2: Pick a Search Strategy**
  - The next step is to choose an appropriate search strategy.
  - The choice of strategy depends on the problem and the computational resources available.
  - There are two main categories:
    - **Uninformed Search**: are strategies like breadth-first search (BFS) and depth-first search (DFS) explore the state space without prior knowledge
    - **Informed Search**: are strategies like A* use heuristics to guide the search toward the goal state.

State Space Search Algorithm

- **Step 3: Start the Search**
  - Once the state space and search strategy are defined, the search begins from the initial state.
  - The system starts exploring the possible states based on the selected search strategy.
  - The search can either proceed by expanding all nodes at each level (in BFS) or exploring a path to its depth (in DFS).

- **Step 4: Extend the Nodes**
  - During the search, each node is expanded by generating its successor states.
  - These successor states represent the next possible configurations after an action is taken.
  - The search process continues by adding these nodes to the frontier or open list, which tracks all unexplored nodes.

State Space Search Algorithm

- **Step 5: Address State Repetition**
  - In large state spaces, it is common to encounter repeated states.
  - To avoid exploring the same state multiple times, it is crucial to maintain a closed list of visited nodes.
  - This prevents the search from wasting resources on redundant paths and ensures a more efficient search process.

- **Step 6: End the Search**
  - The search concludes when the goal state is reached or when all possible states have been explored.
  - If the search successfully finds the goal state, it returns the path that led to the solution.
  - Otherwise, the system may determine that no solution exists within the defined state space.

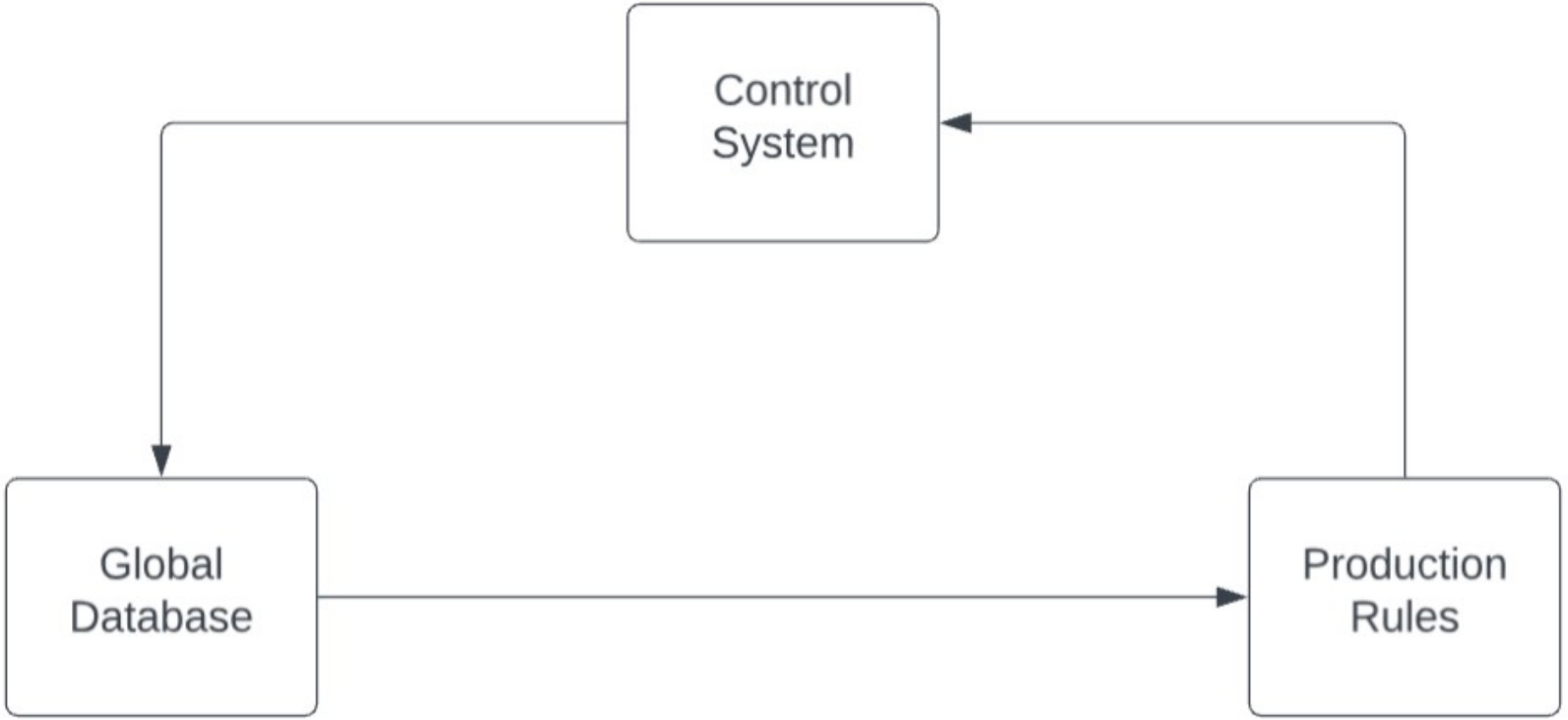
State Space Search Algorithm

- To define a problem as a state space search, you need to identify the following components:
- **State Space**: The set of all possible states of the problem. This includes the initial state, goal state(s), and all the intermediate states.
- **Initial State**: The starting state of the problem.
- **Goal State(s)**: One or more states that represent the desired outcome of the problem.
- **Successor Function**: A function that generates all the possible states that can be reached from the current state by applying an action.
- **Action(s)**: The set of all possible actions that can be taken from a given state.
- **Cost Function**: A function that assigns a cost to each action

Production System

- A production system is based on a set of rules about behavior.
- The production systems in artificial intelligence are rules applied to different behaviors and environment.
- A production system refers to a type of rule-based system that is designed to provide a structured approach to problem solving and decision-making.
- This framework is particularly influential in the realm of expert systems, where it simulates human decision-making processes using a set of predefined rules and facts.
- **Production system or production rule system is a computer program typically used to provide some form of artificial intelligence, which consists primarily of a set of rules about behavior but it also includes the mechanism necessary to follow those rules as the system responds to states of the world.**

Components of Production System



Components of Production System

- The major components of the Production System in Artificial Intelligence are:

- **Global Database**: The global database is the central data structure used by the production system in Artificial Intelligence.

- **Knowledge Base/ Set of Production Rules:** The production rules operate on the global database. Each rule usually has a precondition that is either satisfied or not by the global database. If the precondition is satisfied, the rule is usually be applied. The application of the rule changes the database.

- **A Control System:** The control system then chooses which applicable rule should be applied and ceases computation when a termination condition on the database is satisfied. If multiple rules are to fire at the same time, the control system resolves the conflicts.

Features of Production System

- **Simplicity**: The structure of each sentence in a production system is unique and uniform as they use the "IF-THEN" structure. This structure provides simplicity in knowledge representation. This feature of the production system improves the readability of production rules.

- **Modularity**: This means the production rule code the knowledge available in discrete pieces. Information can be treated as a collection of independent facts which may be added or deleted from the system with essentially no deleterious side effects.

- **Modifiability**: This means the facility for modifying rules. It allows the development of production rules in a skeletal form first and then it is accurate to suit a specific application.

- **Knowledge-intensive**: The knowledge base of the production system stores pure knowledge. This part does not contain any type of control or programming information. Each production rule is normally written as an English sentence; the problem of semantics is solved by the very structure of the representation.

Examples of AI Problems: Tic – Tac – Toe

- Tic-Tac-Toe is a classic game often used to illustrate concepts in Artificial Intelligence, especially state space search.

- The goal of the game is to get three marks in a row (either "X" or "O") on a 3x3 grid, either horizontally, vertically, or diagonally.

- Let's break down how Tic-Tac-Toe can be represented as a state space search problem.
- **Game Representation**
  - A Tic-Tac-Toe game can be represented as a state in the form of a **3 x 3 grid**.
  - Each cell in the grid can be:
    - Empty (denoted by "_")
    - X (player 1's mark)
    - O (player 2's mark)
- For example, the initial state of the game (empty board) looks like this:

  _ _ _

  _ _ _

  _ _ _

Examples of AI Problems

- The state space of Tic-Tac-Toe is the set of **all possible board configurations** that can be achieved during the course of the game.
- The game involves alternating moves between the two players, "X" and "O", until one player wins or the game ends in a draw.
  - Initial State: The game starts with an empty board.
  - Goal States: The goal is to find a configuration where either "X" or "O" has three marks in a row (horizontally, vertically, or diagonally).
- **Terminal States**: The game ends in either a win for one player or a draw (if no more moves are possible and no one has won).
- **Moves**: Players take turns placing their mark ("X" or "O") in an empty cell.

Examples of AI Problems

- **State Representation**
- Each state in the state space represents a specific configuration of the board. For example:
  - Initial State:

    _ _ _
    _ _ _
    _ _ _
  - State after Player 1 (X) makes a move:

    X _ _
    _ _ _
    _ _ _
- State after Player 2 (O) makes a move:

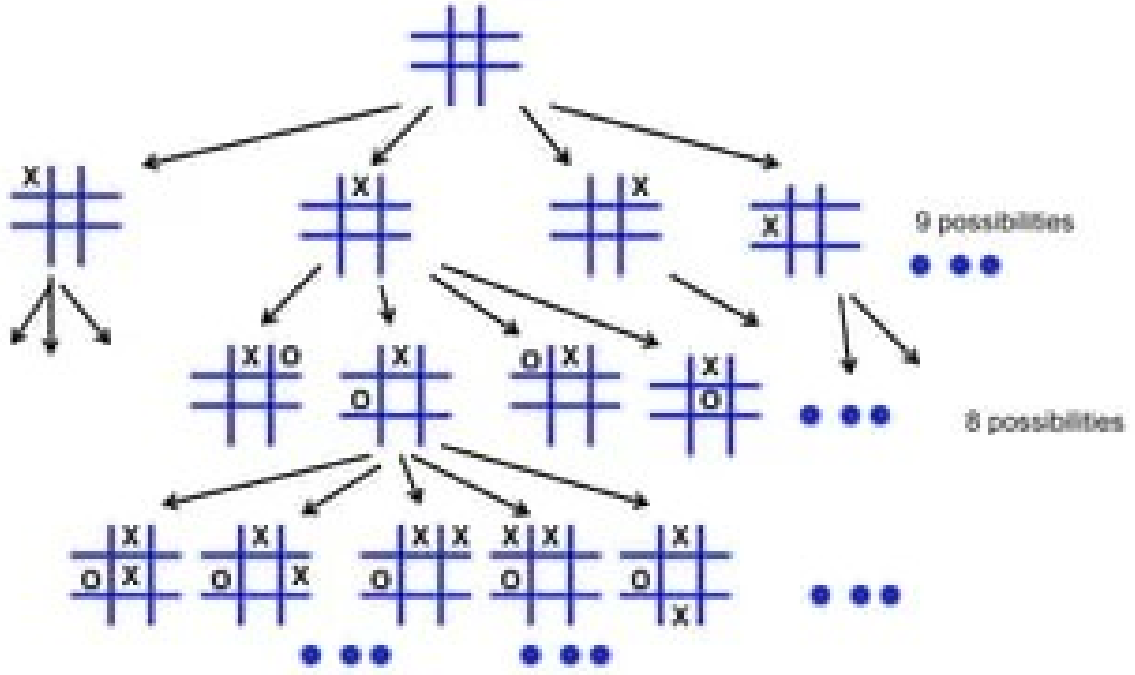
  X _ _
  _ O _
  _ _ _

Examples of AI Problems

- **Actions**
  - At any point in the game, the action is the move made by the current player.
  - The actions are the placement of "X" or "O" in one of the empty cells.
  - For instance, if it's "X"'s turn, the possible actions are the set of empty cells where "X" can be placed.

Examples of AI Problems

- **Actions**
  - At any point in the game, the action is the move made by the current player.
  - The actions are the placement of "X" or "O" in one of the empty cells.
  - For instance, if it's "X"'s turn, the possible actions are the set of empty cells where "X" can be placed.



Examples of AI Problems: Water Jug Problem

- The Water Jug Problem highlights the application of AI to real-world puzzles by breaking down a complex problem into a series of states and transitions that a machine can solve using an intelligent algorithm.
- The Water Jug Problem typically involves two jugs with different capacities.
- The objective is to measure a specific quantity of water by performing operations like filling a jug, emptying a jug, or transferring water between the two jugs.
- **Problem Representation**
  - You are given two jugs with known capacities:
    - JUG 1 – 4 litres
    - JUG 2 – 3 litres
  - Initially, both jugs are empty.
  - The task is to use these two jugs to measure exactly a specific amount of water, such as 2 litres.

Examples of AI Problems: Water Jug Problem

- **Rules:**
  - You can fill either jug completely from a well (or faucet).
  - You can empty either jug at any time.
  - You can pour water from one jug into the other until one of the jugs is either full or empty.
- **Solution Approach:**
  - The Water Jug problem is an example of a state-space search problem.
  - **Define States:** A state in this problem can be represented as a pair of numbers (x, y)(x,y), where xx is the amount of water in the first jug, and yy is the amount of water in the second jug.
  - **Initial State:** The starting state is (0,0) (0,0), where both jugs are empty.
  - **Goal State:** The goal state is any state where one of the jugs contains 2 litres of water (e.g., (2,0)(2,0) or (0,2)(0,2) if the goal is to measure 2 gallons).

Examples of AI Problems: Water Jug Problem

- **Possible Operations**:
  - Fill Jug 1: Fill the first jug completely.
  - Fill Jug 2: Fill the second jug completely.
  - Empty Jug 1: Empty the first jug.
  - Empty Jug 2: Empty the second jug.
  - Pour water from Jug 1 to Jug 2: Pour as much water from the first jug into the second jug until either the first jug is empty or the second jug is full.
  - Pour water from Jug 2 to Jug 1: Pour as much water from the second jug into the first jug until either the second jug is empty or the first jug is full.

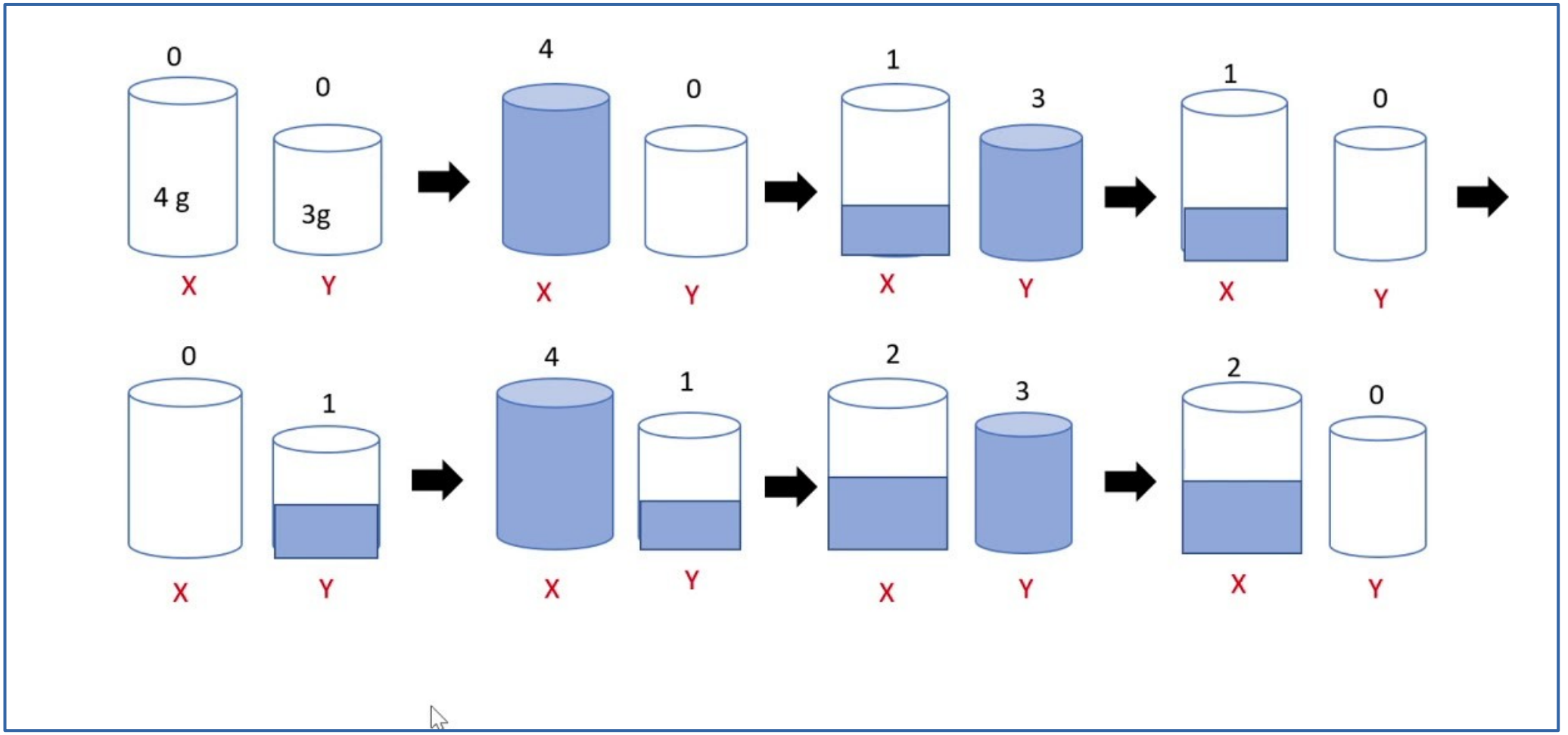
Examples of AI Problems: Water Jug Problem

- Example Solution Using BFS (for 4-gallon and 3-gallon jugs to measure 2 gallons):
  - Initial state: (0,0)(0,0) — both jugs are empty.
  - Fill the 3-gallon jug: (0,3)(0,3).
  - Pour from the 3-gallon jug into the 4-gallon jug: (3,0)(3,0) — now, 3 gallons are in the 4-gallon jug.
  - Fill the 3-gallon jug again: (3,3)(3,3).
  - Pour from the 3-gallon jug into the 4-gallon jug until it is full: This leaves 2 gallons in the 3-gallon jug.
  - Final state is (4,2)(4,2), and we have successfully measured 2 gallons.
  - Thus, the steps to get to the goal would be:
  - (0, 0) → (0, 3) → (3, 0) → (3, 3) → (4, 2)

Examples of AI Problems: Water Jug Problem

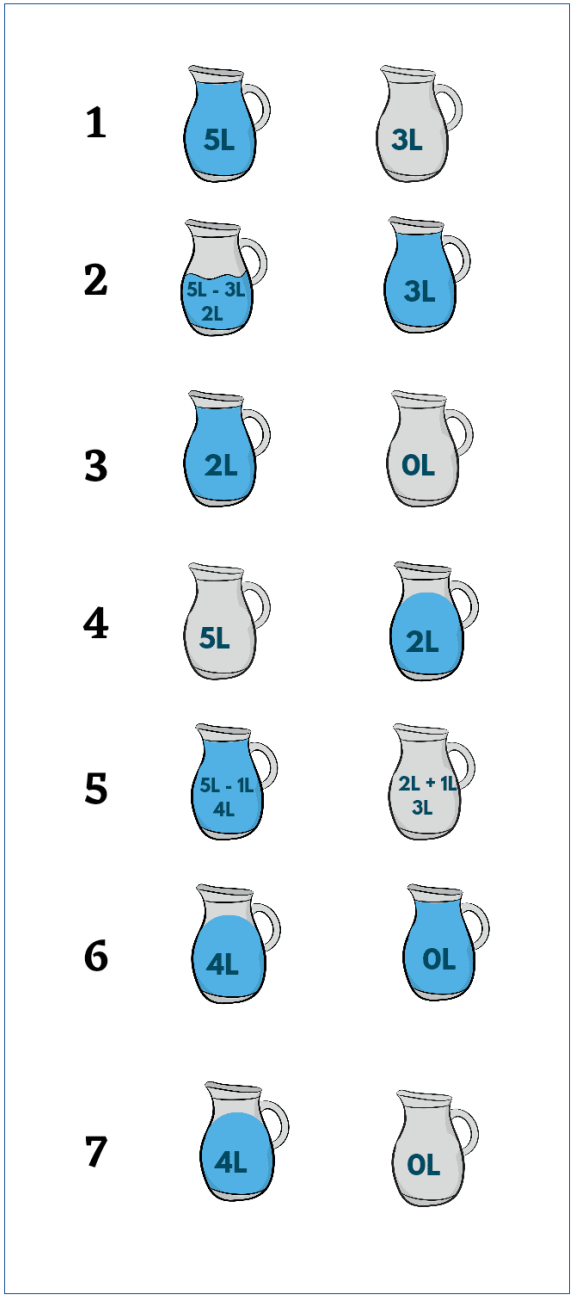
- **Possible Operations**:
  - Fill Jug 1: Fill the first jug completely.
  - Fill Jug 2: Fill the second jug completely.
  - Empty Jug 1: Empty the first jug.
  - Empty Jug 2: Empty the second jug.
  - Pour water from Jug 1 to Jug 2: Pour as much water from the first jug into the second jug until either the first jug is empty or the second jug is full.
  - Pour water from Jug 2 to Jug 1: Pour as much water from the second jug into the first jug until either the second jug is empty or the first jug is full.

Examples of AI Problems: Water Jug Problem



Examples of AI Problems: Water Jug Problem

- JUG 1: 5 ltr.
- JUG 2: 3 ltr.
- Target Measure: 4 ltr.

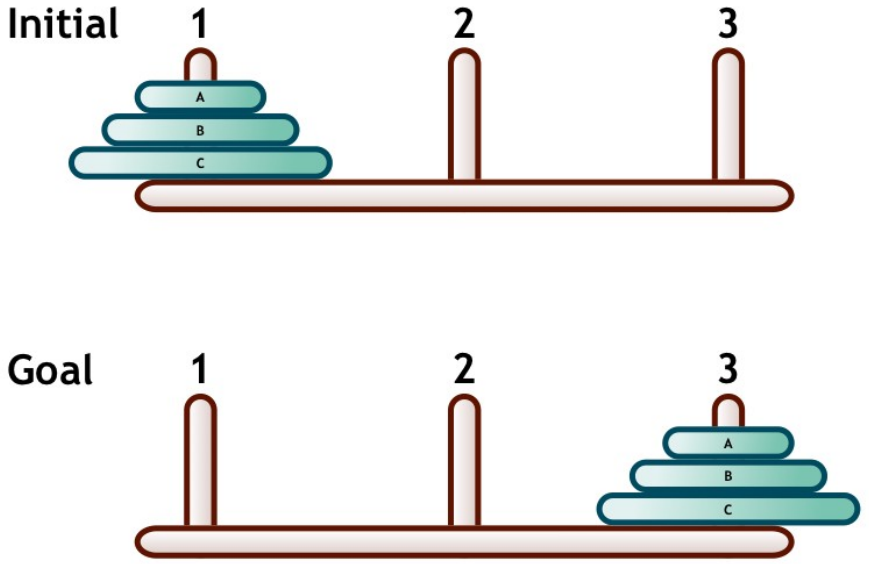


Examples of AI Problems: Water Jug Problem

- Tic – Tac – Toe
- Water – Jug Problem
- Tower of Hanoi
- Traveling Salesman Problem

Examples of AI Problems: Tower of Hanoi

- The Tower of Hanoi problem is a classic mathematical puzzle, invented by the French mathematician Édouard Lucas in 1883.
- There are **3 rods** and **N number of disks** of different sizes.
- All disks are initially placed on the Source rod in increasing order of size (largest at the bottom).
- The smallest disk is at the top, thus making a conical or a pyramid shape.
- **Rules of the Problem**
  - Only one disk can be moved at a time
  - A disk can be placed only on a larger disk or an empty peg
  - All disks must be moved from Source rod to Destination rod



Examples of AI Problems: Tower of Hanoi

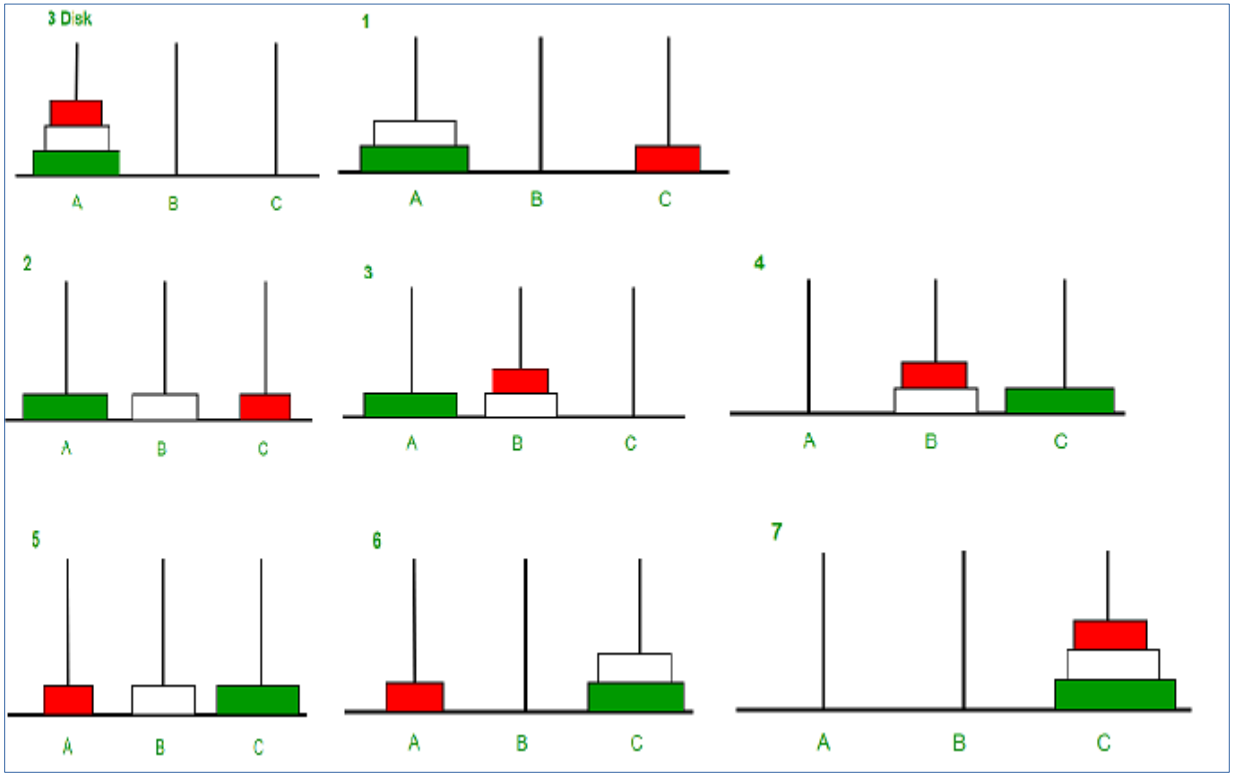
- **Initial Setup:**
  - **Rods**: There are three rods – A, B, and C.
  - **Disks**: A set of disks, each of different sizes. Initially, these disks are stacked on Rod A in descending order with the largest disk at the bottom and the smallest at the top.

- **Constraints | Rules:**
  - Single Move: Pick and move only one disk at a time.
  - Top Disk: You can only move the top disk from any of the three rods.
  - Placement: A disk can only be placed on top of another disk if it is smaller than the disk already on the rod. Alternatively, you can place the disk on an empty rod.
  - Goal: The objective is to move the entire stack of disks from Rod A to Rod C, maintaining their size order.

- **End State:**
  - All n disks are stacked on Rod C in descending order of size, with the largest disk at the bottom and the smallest at the top.

Examples of AI Problems: Tower of Hanoi

- Move disk 1 from A → C
- Move disk 2 from A → B
- Move disk 1 from C → B
- Move disk 3 from A → C
- Move disk 1 from B → A
- Move disk 2 from B → C
- Move disk 1 from A → C



Tower of Hanoi: Applications

Application Area	How Tower of Hanoi Principles Are Applied
Computer Data Storage	The sequence of moves in the Tower of Hanoi is used to determine the optimal rotation schedule for backup tapes, ensuring that older backups are overwritten in a systematic manner.
Robotics	Robots handling stacked objects use the Tower of Hanoi's rules to determine the optimal sequence of moves, ensuring no larger object is placed on a smaller one.
Computer Algorithms	The Tower of Hanoi is directly implemented to teach recursion, demonstrating how a complex problem can be broken down into simpler, repetitive tasks.
Manufacturing	In assembly lines where components need to be stacked or arranged in a specific order, the Tower of Hanoi's principle of moving items without placing a larger one on a smaller one can guide the sequence of operations to avoid mistakes and rework.
Telecommunications	In hierarchical network designs, the Tower of Hanoi's recursive approach can be used as a model to optimize the sequence of connecting nodes or switches, ensuring that higher-capacity nodes (analogous to larger disks) handle more traffic and lower-capacity nodes (smaller disks) handle less.
Logistics	In warehousing, when items are stored in bins or racks, the Tower of Hanoi principle can guide the arrangement such that items are easily accessible. For instance, frequently accessed items (analogous to smaller disks) can be placed in more accessible locations, while less frequently accessed items (larger disks) can be placed deeper or higher up.

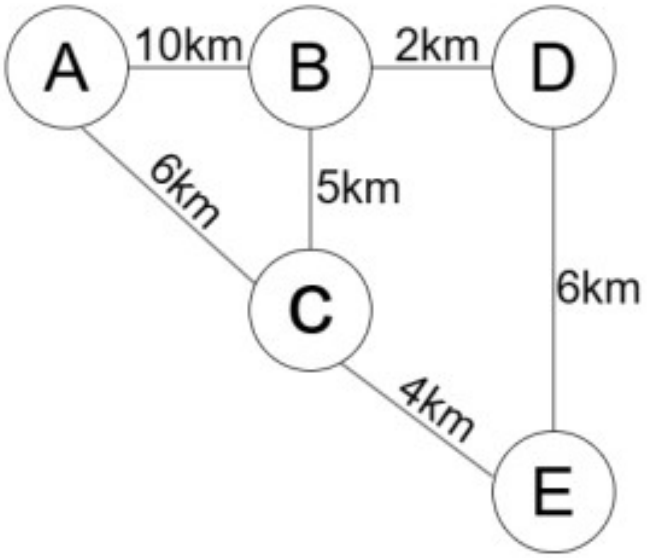
Travelling Salesman Problem

- The **Travelling Salesman Problem** is a graph computational problem.
- Conditions:
- The salesman needs to visit all cities (represented using nodes in a graph) in a list just once and the distances (represented using edges in the graph) between all these cities are known.
- The solution that is needed to be found for this problem is the **Shortest Possible Route** in which the salesman visits all the cities and returns to the origin city.

Solving Travelling Salesman Problem using Simple Approach

- In the following approach, we will solve the problem using the steps mentioned below:
- Step 1: Firstly, we will consider City 1 as the starting and ending point.
- Since the route is cyclic, any point can be considered a starting point.
- Step 2: As the second step, we will generate all the possible permutations of the cities, which are (n-1)!
- Step 3: After that, we will find the cost of each permutation and keep a record of the minimum cost permutation.
- Step 4: At last, we will return the permutation with minimum cost.

Solving Travelling Salesman Problem using Simple Approach



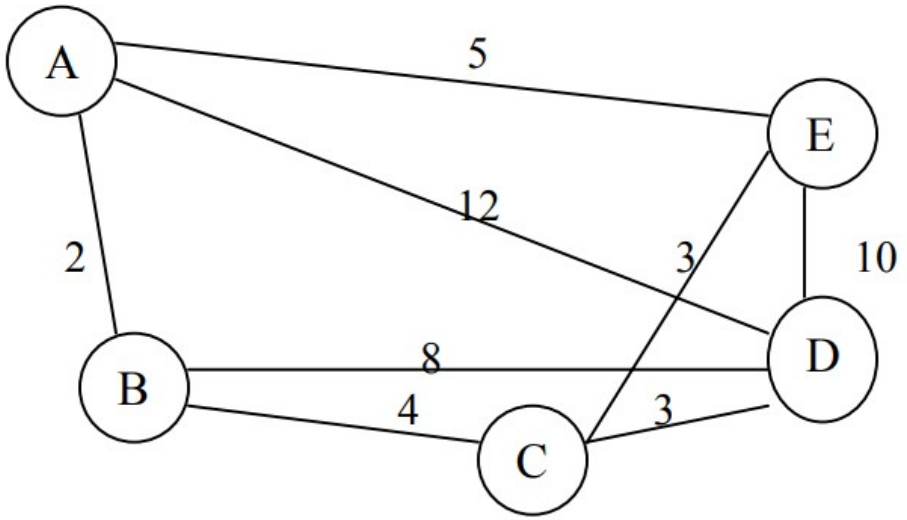
Goal: City "A" to "E"

Path1 : A – B – D – E
10 + 2 + 6 = 18

Path2 : A – C – E
6 + 4 = 10

Path3: A – B – C – E
10 + 5 + 4 = 19

Solving Travelling Salesman Problem using Simple Approach

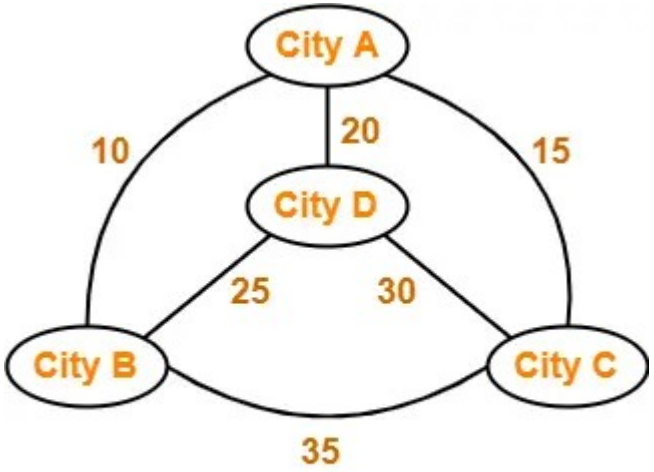


GOAL: Start from A and come back to A

Path1 : A– B – C – D – E – A
2 – 4 – 3 – 10 – 5 = 24

Path2 : A – B – C – E – D – A
2 – 4 – 3 – 10 – 12 = 31

Solving Travelling Salesman Problem using Simple Approach



Goal: City "A" to "A"

Path1 : A – B – C – D – A

10 + 35 +30 + 20 = 95

Path2 : A – B – D – C – A

10 + 25 + 30 + 15 = 8